

Intro to Ilm-d and PD Disaggregation



llm-d in nutshell

- **Core Purpose:** Scales LLM inference on Kubernetes.
- **Tech Stack:** Uses vLLM and Inference Gateways.
- **Key Backers:** Built by Red Hat, Google, IBM and Intel
- **Main Feature:** Routes requests via KV-cache awareness.
- **GitHub Layout:** Split into modular, open-source repositories.

llm-d architecture

Client

Kubernetes

GET /completions

Inference Gateway
(e.g. Envoy)

Route to
selected pods

Select *InferencePool*
from model name
(OAI spec)

**Body-based
Routing**

Select optimal model
replica based on state

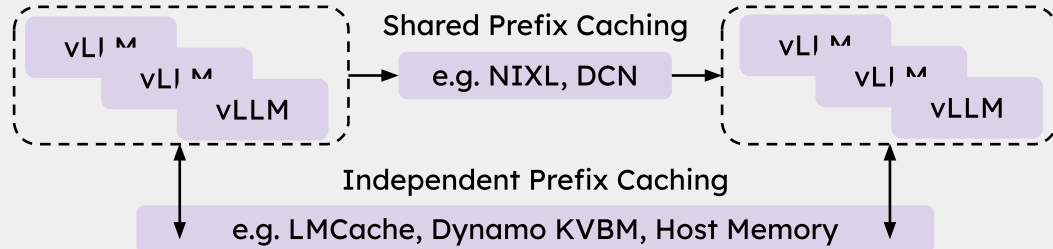
**Inference
Scheduler**

Extensible library of
scrapers, scorers,
filters for load-, KV-
and P/D- aware routing

Inference Pool

Variant A (e.g. Prefill)

Variant B (e.g. Decode)



Load, KV
Cache Report

**Variant
Autoscaler**

Translate capacity
bounds, saturation
measurements,
and traffic mix to
variant count

Update
replicas

Nodes



What is PD Disaggregation?

Problem: In standard LLM inference, every request does two things in the same GPU process:

1. **Prefill** - process all input tokens, build KV cache (compute-heavy, memory-write)
2. **Decode** - generate output tokens one-by-one using that KV cache (memory-bandwidth-heavy)

These two phases have opposite bottlenecks.

Solution: Run them on separate containers/GPUs.

Why PD Disaggregation?

Benefits

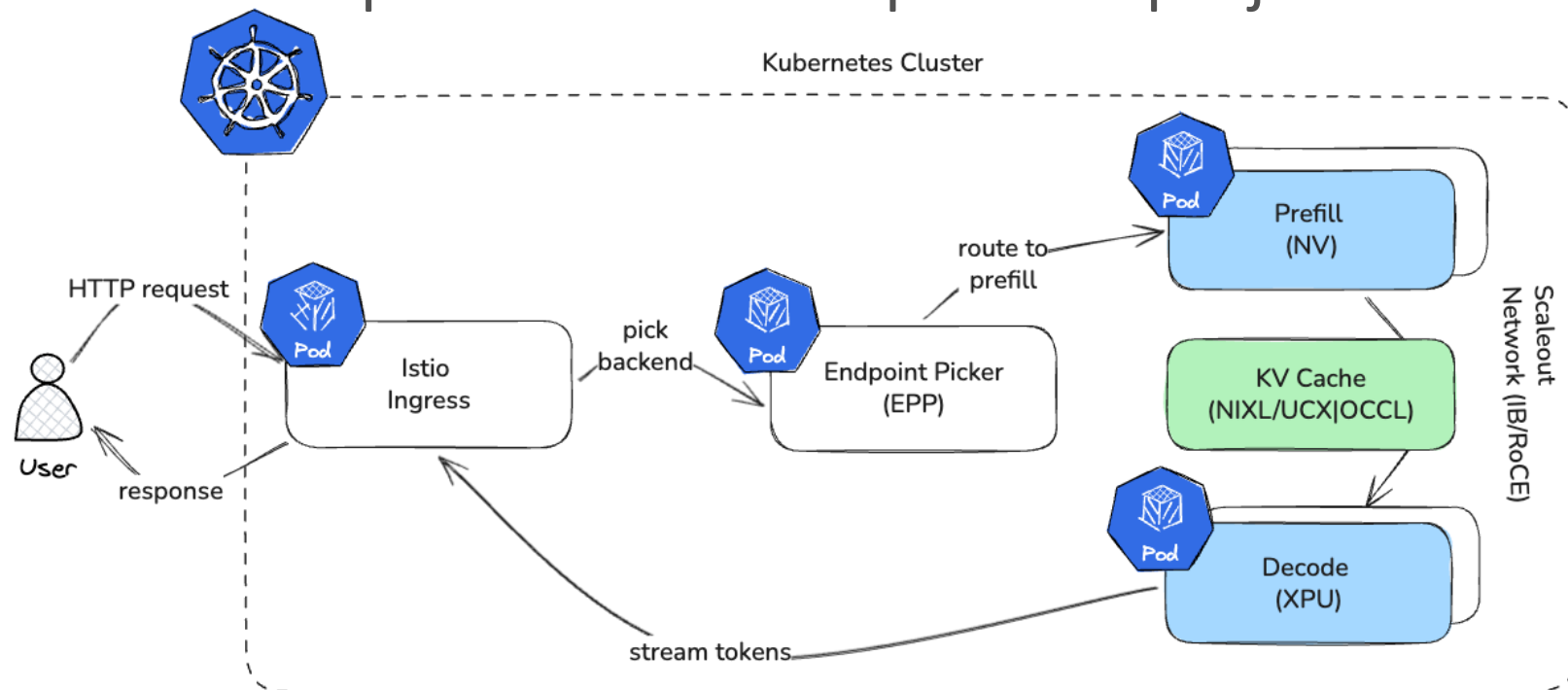
- Prefill and decode scale independently
- Lower TTFT (time to first token)
- Higher decode throughput (no prefill interference)
- Eliminates Resource Interference: In unified systems, a large prefill request can "block" the decode phase of existing requests, causing spikes in Inter-Token Latency (ITL).

Performance Proofpoints

- Few industry reports indicating 5-10x performance gains in large scale deployments.

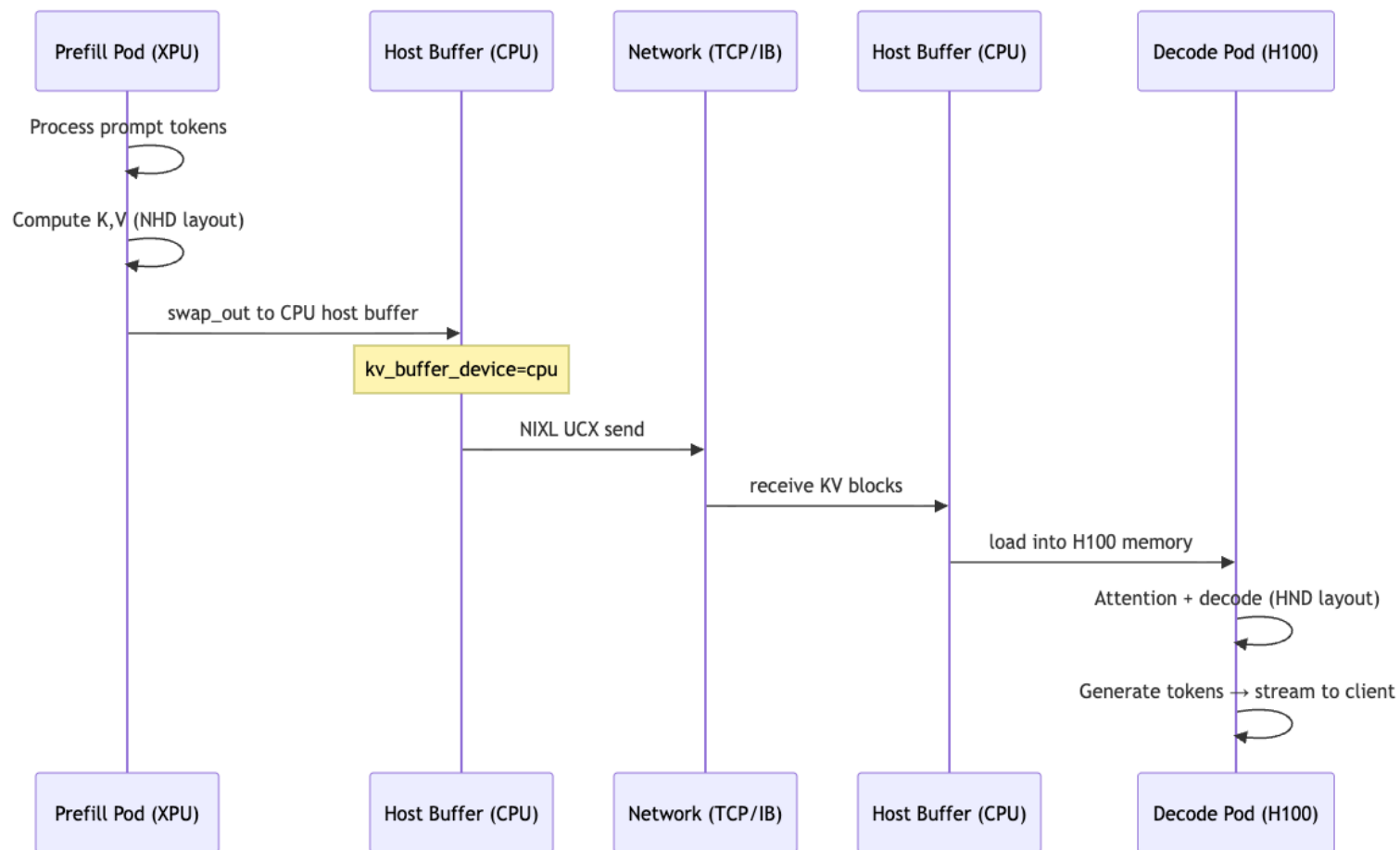
Ilm-d System Architecture

Ilm-d is built on top of Kubernetes upstream projects.

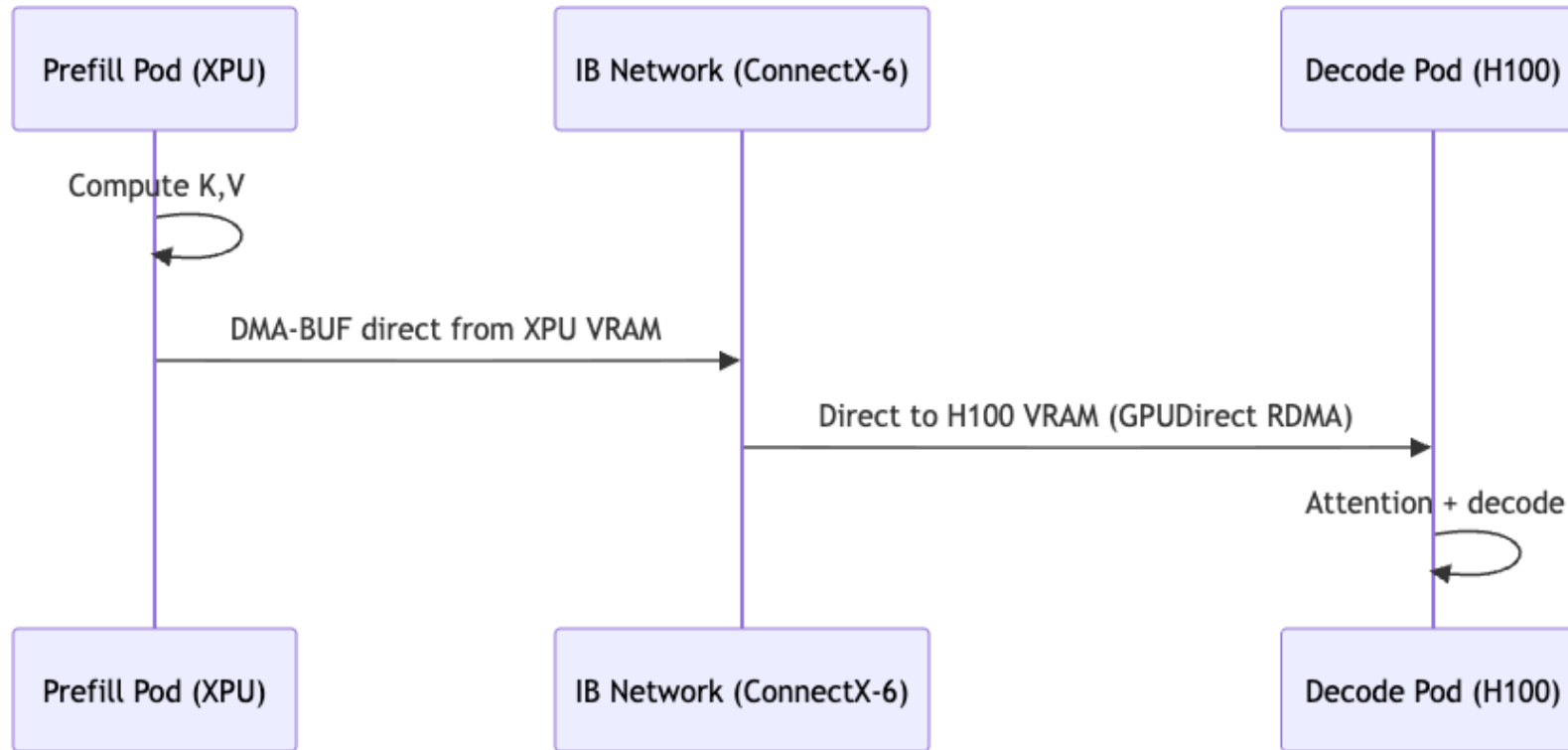


Block	Function	Upstream Project
Istio Ingress	Ingress, TLS, traffic management	Istio / Kubernetes Gateway API
Endpoint Picker (EPP)	Prefix-cache-aware pod selection	Gateway API Inference Extension
Inference Pods (Prefill/Decode)	vLLM prefill + decode on accelerators	vLLM

KV Cache and NIXL Transfer (tcp/rdma)



KV Cache and NIXL Transfer (GPU Direct)



Demo

Component	Details
Cluster	k3s v1.35
Intel XPU nodes	pvc1–pvc8 — Intel Data Center GPU Max 1550
IB Network	Mellanox ConnectX-6 on all nodes, DRANET DRA driver
Storage	models pre-cached on NFS server
llm-d version	v0.6.0
Model	Qwen/Qwen3-0.6B

- XPU - XPU PD Disaggregation with RDMA
- 3 PODs (P, D, EPP)
- Terminal 1: relevant logs from P and D pods
- Terminal 2: LLM request and response, and relevant statistics